git

1. Introduction to Version Control

Version control is a system used to record changes to files over time, allowing multiple developers to work together efficiently. It provides:

- **Tracking of changes** made to a project.
- **Collaboration** by enabling multiple people to work on the same codebase.
- **Reverting** to earlier versions in case of mistakes.
- **Branching and merging**, allowing parallel lines of development.

Version control systems (VCS) are essential tools that track and manage changes to files over time. They help maintain a complete history of modifications, enabling developers to compare versions, identify mistakes, and restore previous states. VCS supports collaboration by allowing multiple developers to work simultaneously without overwriting each other's changes. There are two main types of VCS:

- **Centralized VCS (CVCS):** A central server stores the versioned files. Example: SVN. The drawback is dependency on a single server.
- **Distributed VCS (DVCS):** Every developer has a complete copy of the repository, including full history. Example: Git. This ensures speed, redundancy, and offline availability.

Git is a DVCS and is highly preferred due to its branching model, performance, and flexibility.

Git is a distributed version control system, meaning every developer has a full copy of the project history.

2. Basic Concepts of Git

Repository Types

- **Local Repository:** Stored on your machine; contains your commits and branches.
- **Remote Repository:** Stored on servers (GitHub, GitLab, Bitbucket). Used for collaboration.

Key Terms

- **Commit:** A snapshot of your project at a specific moment.
- **Working Directory:** Files you are currently editing.
- **Staging Area (Index):** Temporary area where changes are prepared for committing.
- **HEAD:** Pointer to the latest commit in the current branch.

Git Workflow

1. Modify files in the working directory.
2. Stage changes using git add.
3. Commit changes using git commit.

Git uses a unique data model where changes are stored as snapshots rather than file differences. Each commit creates a new snapshot of the project and links it to previous commits, forming a Directed Acyclic Graph (DAG). Git uses checksums (SHA-1 hashes) to ensure integrity—meaning files cannot be changed without Git knowing. Git also uses three key states:

- **Modified:** File has changed but not staged.
- **Staged:** File is ready to be committed.
- **Committed:** File changes are safely stored in Git history.

Git's distributed model allows every clone to act as a full backup.

3. Setup Git Repository

Initialize a New Repository

git init

Creates a new Git repository in the current folder.

Clone an Existing Repository

git clone <repo-url>

Downloads the entire project with its history.

Configure Git (One-time setup)

git config --global user.name "Your Name"

git config --global user.email "your@email.com"

This information appears in your commits.

Initializing a Git repository converts a normal folder into a tracked project by creating a .git directory. This directory stores:

- Commit objects
- Branches and tags
- Configuration files
- Staging area information

Cloning from a remote repository creates a full local copy, enabling independent work until pushed back to the remote.

4. Working With Git

The Git workflow moves through three areas:

- **Working Directory:** where actual code editing happens.
- **Staging Area:** a buffer where developers prepare specific changes.
- **Repository (Local):** final stored commits.

Working with Git involves understanding how Git tracks changes using blobs (file contents), trees (directory structures), and commits (project snapshots). Git also stores references like HEAD to track the current branch.

Track Changes

git status

Shows modified, staged, and untracked files.

Stage Changes

git add <file>

git add .

Commit Changes

git commit -m "Meaningful commit message"

A good commit message describes what and why.

Push Changes to Remote

git push origin <branch-name>

Pull Latest Changes

git pull

Fetches and merges changes from the remote repository.

5. Concept of Branches

Branches allow creating isolated development environments.

- **main/master:** Stable production-ready code.
- **feature branches:** For new features.
- **hotfix branches:** For urgent bug fixes.

Commands

git branch <name>

git checkout <name>

git checkout -b <name>

6.Merge Requests (Pull Requests)

Merge Requests (GitLab) or Pull Requests (GitHub) create a collaboration workflow where:

- Code is pushed to a separate branch.
- Team members review the code.
- Automated tests run.
- The maintainer approves and merges.

It ensures code quality, reduces bugs, and encourages teamwork through discussion threads and review comments.

Used in platforms like GitHub/GitLab to:

- Propose merging a branch.
- Allow team members to review code.
- Maintain code quality.

Developers create a PR/MR when a feature is complete.

7. Deleting Branches

Branches can be deleted locally and remotely after merging to keep the repository clean. Deleting a branch doesn't affect Git history because commits still remain unless a prune is performed. Remote branches require explicit deletion because Git doesn't automatically clean them.

Delete Local Branch

git branch -d <name>

Use -D to force delete.

Delete Remote Branch

git push origin --delete <name>

8. Rebase

Rebase helps maintain a linear project history by moving or replaying commits from one branch onto another. Unlike merging, rebase avoids unnecessary merge commits. However, rebase rewrites commit history, which can be dangerous for branches already pushed to shared repositories.

Rebase is ideal for:

- Cleaning up messy commit histories.
- Creating a smooth and readable project timeline.

Rebase rewrites commit history by placing your commits on top of another branch.

git rebase <branch>

Benefits

- Cleaner, linear commit history.

Risks

- Should not be used after pushing to shared branches.

9. Resolving Merge Conflicts

Conflicts occur when Git cannot automatically merge two versions of a file. Conflict resolution requires understanding the structure of conflict markers:

<<<<<<< HEAD

Your current branch's changes

=======

Incoming branch's changes

>>>>>>> other-branch

After manually resolving, Git requires staging the fixed file to proceed. Merge conflicts often arise in files where multiple developers make overlapping changes.

A merge conflict happens when Git cannot automatically merge changes.

Steps to Resolve

1. Open the file showing conflict markers (<<<<, ====, >>>>).
2. Choose which changes to keep or manually combine.
3. Stage the resolved file:

git add <file>

4. Continue merge or rebase.

10. .gitignore

The .gitignore file tells Git which files to ignore. These typically include:

- Build outputs

- Dependencies
- Logs
- Sensitive files

Gitignore patterns support wildcards, directory patterns, and negations. Once a file is tracked, adding it to .gitignore won't remove it—you must untrack it manually.

Specifies files and folders Git should not track.

Common entries

node_modules/

.env

*.log

/dist

Useful for preventing sensitive or unnecessary files from entering the repository.

11. Git Stash

Git stash saves your uncommitted changes so you can switch branches or pull updates safely. Stash creates a commit-like object stored in a stack structure. Developers often use stash for temporary work, especially when interrupted by urgent tasks.

Temporarily stores uncommitted changes without committing them.

Commands

git stash

git stash list

git stash apply

git stash drop

Useful when switching branches without losing progress.

12. Going Back to History

Git stores complete project history, allowing developers to:

- Review the evolution of the project
- Audit code changes
- Identify bugs via tools like git bisect

Because of the immutable nature of commits, Git ensures historical integrity and accountability.

View commit history

git log

For shorter view:

git log --oneline

See the changes in a commit:

git show <commit-id>

13. Undoing Commits

Undoing history depends on whether commits are pushed or not:

- **Reset** rewrites history (dangerous for shared repos).
- **Revert** creates new commits to safely undo changes.
- **Restore** helps recover earlier file versions.

Understanding when to use reset vs revert is crucial for team workflows.

Soft Reset (keep files)

git reset --soft HEAD~1

Hard Reset (remove changes)

git reset --hard HEAD~1

Revert Commit

git revert <commit-id>

Safe for undoing commits in shared branches

14. Merging Branches

Git merge combines histories. Two major types:

- **Fast-forward merge:** Simple pointer movement.
- **Three-way merge:** Combines commits from diverged branches.

Merge commits help track integration points and preserve branch context

Merge Process

1. Switch to the target branch (e.g., main):

git checkout main

2. Merge feature branch:

git merge feature-branch

Types of Merges

- **Fast-forward merge:** No conflicts; linear history.
- **Three-way merge:** Creates a merge commit.

15. Git for DevOps

Git is fundamental in DevOps workflows:

- **CI/CD pipelines:** Automatically build, test, deploy using tools like GitHub Actions, GitLab CI.
- **Infrastructure as Code (IaC):** Manage infrastructure using version control tools.
- **Collaboration & automation:** Helps teams release software faster and more reliably.
- **Versioning of scripts, pipelines, configs.**

Git ensures consistency, traceability, and rollback ability in deployments.

17. Databases in Development Process

Databases store and manage application data.

Types of Databases

- **SQL (Relational):** MySQL, PostgreSQL, Oracle. Store data in tables.
- **NoSQL (Non-relational):** MongoDB, Redis, Cassandra.

Importance in Development

- Maintain user accounts, sessions, transactions.
- Power backend services.
- Store app configuration, logs, analytics.

Key Concepts

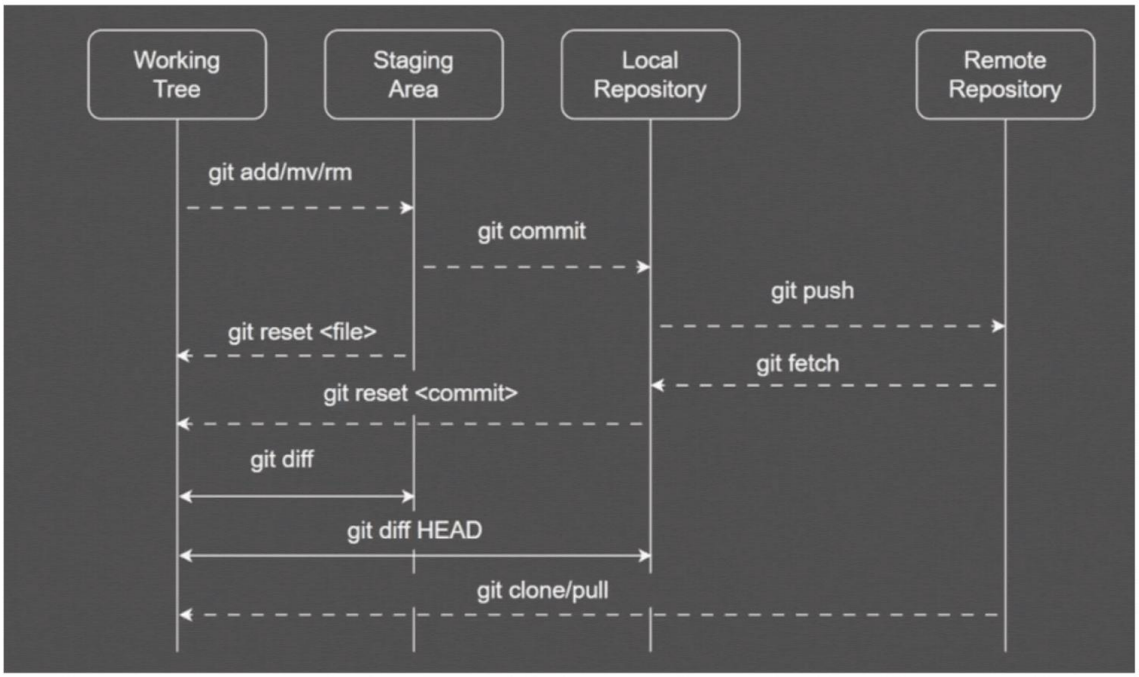
- **Tables/Collections:** Structure for storing data.
- **Primary Keys:** Unique identifiers.
- **Foreign Keys:** Relationships between tables.
- **Indexes:** Improve search speed.
- **Normalization:** Reduce data duplication.
- **Transactions:** Ensure data integrity.

Database Development Workflow

1. Design schema.
2. Connect database to backend.
3. Implement CRUD operations.
4. Use migration tools (Flyway, Liquibase) to version database changes.
5. Optimize using indexing and query tuning.
6. Backup, restore, and monitor performance.

These notes now provide detailed, comprehensive explanations suitable for study and revision.

Git Workflow



Branching Model Diagram

```
main ————————●————————●————————●————————●————————
             \                  \
              \                  ● feature-2
               \
                ● feature-1 ——●——● merge
```

Merge vs Rebase Diagram

Merge

```
A: ●──●──●──────────────────●
       \        /
B:       ●──●──●──● merge commit
```

Rebase

```
A: ●──●──●──────────────────────●
           replay B commits
B:    ●──●──●──●──────────────┘
```

DevOps CI/CD Pipeline Diagram

Developer → Git Push → CI Server → Build → Test → Deploy → Monitoring

Real-World Examples

Version Control Example

A team building an e-commerce site uses separate branches:

- feature/cart
- feature/payment
- feature/search
  Each developer works independently and merges only after completing their module.

Branching Example

You want to add dark mode to your website:

git checkout -b feature/dark-mode

You work on it without affecting the main branch.

Merge Conflict Example

Two developers edit the same part of header.css. Git can't decide which one to keep — conflict happens.

Rebase Example

Before opening a PR, you rebase your feature branch to keep history clean:

```
git rebase main
```

DevOps with Git Example

GitHub Actions automatically deploys a new version of your website whenever you push to the main branch.

Best Practices

Git Best Practices

- Commit small and often.
- Write meaningful commit messages.
- Use feature branches for new work.
- Pull frequently to avoid conflicts.
- Avoid rebasing shared branches.

Branching Best Practices

- Name branches clearly: feature/login, bugfix/header.
- Delete merged branches to keep the repo clean.

Merge Best Practices

- Always review code before merging.
- Resolve conflicts manually and carefully.

DevOps Best Practices

- Automate tests before deployment.
- Use Git as the single source of truth (GitOps).
- Tag releases using semantic versioning.

Database Best Practices

- Normalize tables to reduce redundancy.

- Use migrations to track schema changes.
- Always index frequently queried columns.

Interview Questions for Each Topic

1.Introduction to Version Control

1. What is version control and why do we use it?
2. Difference between centralized and distributed VCS.
3. Why is Git more popular than SVN?

2.Basic Concepts of Git

1. What is a commit in Git?
2. What is the difference between staging and committing?
3. Explain what a hash (SHA-1) represents.

3.Git Repository Setup

1. What does git init do internally?
2. Difference between git clone and git init.

4.Working With Git

1. What is the Git workflow?
2. Why do we stage changes before committing?

5.Branching Concepts

1. What is a branch and why do we use it?
2. Difference between main, feature, and hotfix branches.

6.Merge Requests

1. What is a PR/MR and why is it important?
2. What happens during code review?

7.Deleting Branches

1. Difference between -d and -D when deleting branches.

8.Rebase

1. What is rebasing? When should you NOT use it?
2. Difference between merge and rebase.

9.Merge Conflicts

1. What causes merge conflicts?
2. How do you manually resolve a conflict?

10. .gitignore

1. Why do we use .gitignore?
2. How to ignore tracked files?

11.Git Stash

1. When do we use stash?
2. Difference between stash apply and stash pop.

12.History Navigation

1. What does git log show?
2. What is HEAD in Git?

13.Undoing Commits

1. Difference between reset and revert.
2. When is hard reset dangerous?

14.Merging

1. What is fast-forward merge?
2. What is a three-way merge?

15.Git for DevOps

1. How does Git support CI/CD?
2. What is GitOps?